

UE5.7 联机 FPS 游戏制作技术文档

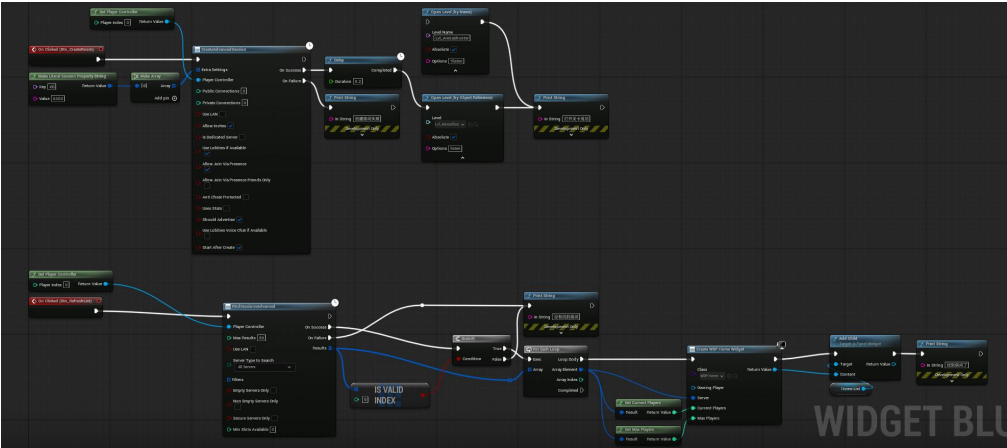
文档说明

本文档基于 Unreal Engine 5.7 开发方案，记录多人对战 FPS 游戏的联机开发流程、网络同步核心逻辑、功能模块实现、问题修复及技术规范，用于复盘查阅及版本迭代参考。

一、项目基础信息

本项目为多人联机 FPS 对战游戏，采用房主服务器架构，适配 2 人联机对局或单人对战 AI 敌人，依托 Steam 实现联机通信。

属性	详情
引擎版本	Unreal Engine 5.7
开发方式	蓝图（Blueprint），少量 C++内容
网络架构	Listen Server（房主即服务器）
联机方案	Steam OSS 子系统，AdvancedSessions
支持玩家规模	1-2 人
游戏类型	FPS 联机对战游戏



P1.会话访问

二、核心技术架构

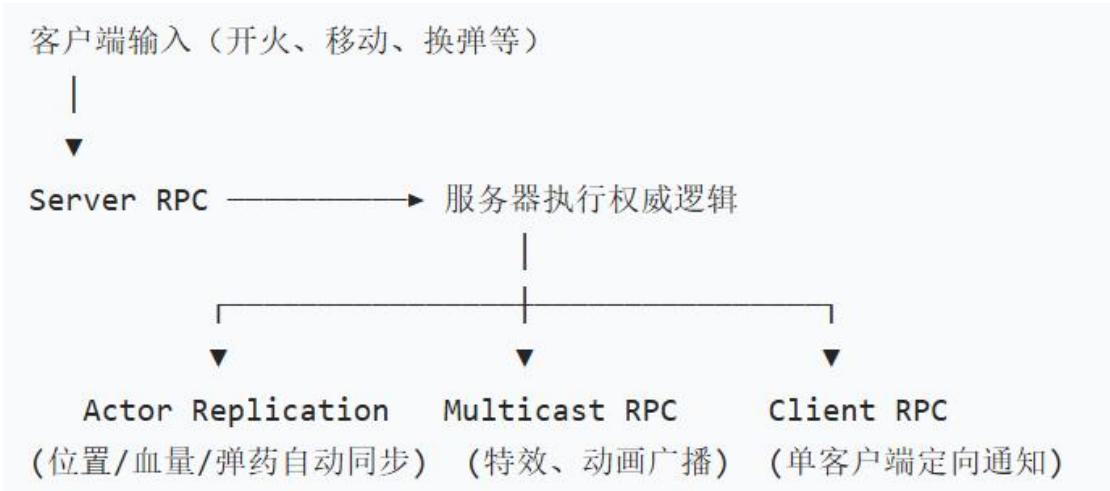
2.1 网络通信分层设计

项目采用逻辑层、表现层双层通信架构，两层完全解耦，数据单向流转，分离业务计算与画面音效展示，保证多人联机同步稳定、模块职责清晰。

网络层级	核心组成模块	层级功能说明
逻辑层	GameMode GameState PlayerState Character WeaponBase Projectile	负责所有核心业务运算，不包含任何特效、音效渲染逻辑；处理开火判定、弹道校验、命中检测、伤害数值计算、玩家得分统计、对局规则、胜负结算。依靠 Actor Replication、Server RPC、Multicast RPC、RepNotify 完成跨端数据同步，所有操作与数值计算统一在服务端校验，同步后仅对外输出状态信号，不直接驱动画面效果。
对象层	Widget 各类特效组件 音效播放	仅负责视听效果展示，无伤害、命中、规则等业务计算能力；接收逻辑层同步下发的事件信号，触发枪口开火特效、开枪音效、UI 状态刷新等表现内容，仅做可视化、音频反馈，不修改游戏核心数据。

2.2 核心数据流方向

项目遵循「客户端输入、服务器权威判定、同步反馈」的标准 FPS 联机数据流，防止客户端本地判定导致的同步异常。



P2.联机数据流结构

2.3 网络同步机制选型

针对不同游戏逻辑场景，匹配专属网络同步机制，兼顾同步精度与运行性能，避免冗余网络开销。

机制	适用场景	示例
Actor Replication	持续变化的状态数据同步	角色位置、血量、弹药量
Server RPC	客户端请求服务器执行核心权威逻辑	开火、换弹、切换武器、请求复活
Multicast RPC	向所有客户端广播视觉、听觉效果	枪口动画、爆炸特效、开枪音效
Client RPC	定向通知单个客户端执行专属操作	登录跳转大厅、个人分数提示
RepNotify	变量同步后触发连锁业务逻辑	血量归零触发死亡逻辑、弹药更新刷新 UI
GameState 轮询	全局公共数据跨客户端同步	队伍分数

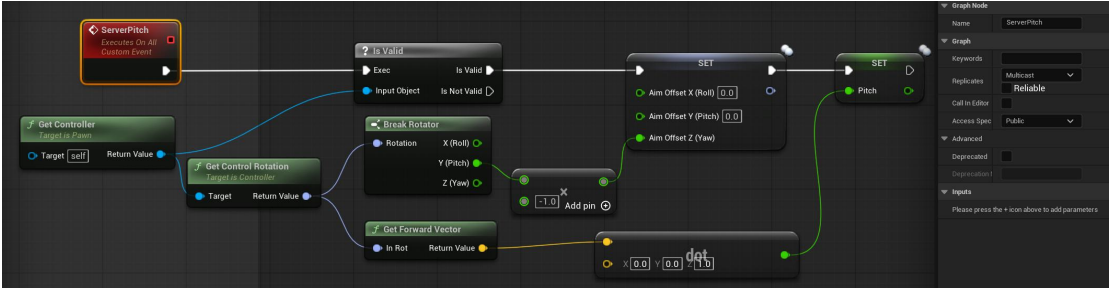
三、核心技术难点深度解析

3.1 动画与网络权限解耦难点

核心原理：初期所有开火逻辑集中在服务器权限分支，导致客户端无任何视觉反馈，违背 FPS 游戏即时交互体验需求。俯仰角只在客户端执行，导致其余客户端无任何视觉反馈。

解决方案：拆分业务逻辑层级，纯视觉的动画、特效、音效逻辑脱离网络权限判断，双端本地独立执行；子弹生成、弹药扣除、伤害判定等权威逻辑仅由服务器执行。俯仰角多播至所有客户端。

开发教训：联机游戏必须严格区分「视觉表现层」与「数据权威层」，解耦开发，兼顾同步正确性与操作流畅度。



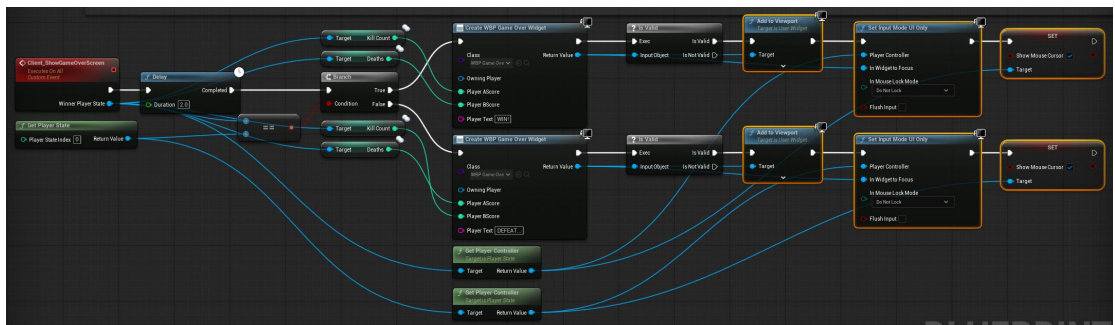
P3.俯仰角联机不显示解决思路

3.2 GameMode 客户端数据共享误区

核心原理：GameMode 为服务器独有实例，仅存在于房主服务器内存中，所有客户端无 GameMode 镜像，无法读取其变量、绑定其委托。

解决方案：全局公共游戏数据全部迁移至 GameState，GameState 为引擎专属跨端同步容器，复制变量可自动推送至所有客户端，支持全局数据实时共享。

开发教训：严格遵循 UE 数据存储规范：GameMode 存服务器规则、GameState 存全局状态、PlayerState 存个人玩家数据。



P4.计数板逻辑（使用 state 存储数据）

3.3 玩家数组时序不可靠问题

核心原理：客户端角色 BeginPlay 执行时机早于 GameState 玩家数组填充时机，此时 PlayerArray 为空，通过数组索引分配队伍的方式必然报错。

解决方案：放弃数组索引匹配方式，改用引擎原生 PlayerId 属性，该属性全局唯一、自动递增，不受游戏时序影响，任意阶段读取均有效。

开发教训：多人联机开发优先使用引擎原生自带网络属性，减少自定义逻辑，规避时序、空值等隐性 BUG。

3.4 角色 Possess 控制权冲突难点

核心原理：UE5 中单个 PlayerController 同一时间仅能 Possess 一个 Pawn 角色，多路径同时生成角色会导致控制权抢占、角色被强制剔除。

解决方案：全局统一角色复活、生成链路，删除所有冗余生成逻辑，确保同一玩家仅存在一套角色生命周期管控逻辑。

开发教训：涉及角色生成、控制权绑定的逻辑，必须全局统一入口，禁止多路径并行执行。

四、核心蓝图文件修改汇总（部分）

蓝图文件	修改性质	核心改动内容
BP_ShooterProjectileBase	弹射物同步优化	开启 bReplicates 复制；碰撞逻辑添加服务器权限判断；伤害判定服务端执行

BP_ShooterCharacter	核心逻辑重构	新增服务器开火 RPC；新增计分逻辑
BP_ShooterWeaponBase	逻辑解耦优化	开火函数设为本地执行；动画节点移出权限分支；开火逻辑拆分双端执行
BP_ShooterGameState	全局数据扩展	新增可复制队伍分数变量，承载对局数据同步
BP_ShooterGameMode	规则逻辑优化	优化玩家复活定时逻辑；清理老旧分数统计接口
BP_ShooterPlayerState	玩家数据同步	击杀、死亡数据开启网络复制，实现战绩同步
BP_ShooterPlayerController	逻辑精简	删除即时复活冗余链路，统一服务端复活逻辑
UI_Shooter	UI 同步重构	改用定时轮询 GameState 实现分数实时刷新

五、项目开发规范与最佳实践

- 1. **网络权限规范**：所有核心权威逻辑（生成 Actor、扣血、扣弹药、伤害判定）仅服务器执行；动画、特效、音效、UI 反馈等视觉逻辑双端本地执行。
- 2. **数据存储规范**：严格区分三层数据容器，GameMode 存服务器对局规则、GameState 存全局同步数据、PlayerState 存玩家个人数据，禁止跨层存储。
- 3. **RPC 使用规范**：仅在已开启网络复制的 Actor 上定义 RPC 事件，非复制对象禁止承载任何网络请求逻辑。
- 4. **视觉逻辑规范**：所有交互反馈类视觉效果脱离网络权限判断，优先保证客户端即时交互体验，同步逻辑后置兜底。
- 5. **生命周期规范**：角色生成、复活、销毁、控制权绑定逻辑必须统一唯一入口，杜绝多路径并行执行引发的同步冲突。